

Rybina search (rybina)

À la Schueberfouer, la plus grande fête foraine du Luxembourg, il existe un jeu de devinettes très populaire. Le forain, Rybina, cache un mot de passe secret sous la forme AB , où A et B sont des chaînes de N chiffres. Il vous permet de deviner le mot de passe et, après chaque essai, il vous indique si votre proposition est trop haute ou trop basse.

Cependant, après chaque essai, Rybina peut échanger les deux moitiés du mot de passe, c'est-à-dire que AB devient BA (ou BA devient AB). Il vous précisera également s'il l'a fait. Votre prochaine tentative sera alors comparée au mot de passe inversé.

Vous devez trouver le mot de passe secret en utilisant au maximum 64 essais.

Implémentation

Vous devrez soumettre un seul fichier source `.cpp`.



Parmi les pièces jointes de cette tâche, vous trouverez un modèle `rybina.cpp` avec un exemple d'implémentation.

Vous devrez implémenter la fonction suivante :

C++

```
void rybina_search(int N)
```

- L'entier N représente le nombre de chiffres dans chaque moitié du mot de passe.

Votre programme peut appeler la fonction suivante, qui est déjà définie dans l'évaluateur (grader) :

C++

```
GuessResult guess(long long X)
```

- L'entier X représente le mot de passe que vous avez saisi comme tentative. Il doit être positif ou nul et comporter au plus $2N$ chiffres lorsqu'il est écrit en décimal (c'est-à-dire $0 \leq X < 10^{2N}$).
- Si votre tentative est correcte, votre programme s'arrête et vous recevez le verdict **accepted** pour le cas de test actuel.
- Sinon, la fonction renvoie un `GuessResult`, une structure avec un champ entier `comparison` et un champ booléen `swapped`.
- La valeur de `comparison` est 1 si votre tentative était trop basse et -1 si elle était trop haute.
- La valeur de `swapped` indique si le mot de passe a été inversé après que votre tentative a été faite.

Notez que l'évaluateur est **adaptatif** et que la réponse correcte ou les inversions pour un cas de test ne sont donc pas fixées à l'avance.

Évaluateur

Parmi les pièces jointes de ce problème, vous trouverez une version simplifiée de l'évaluateur (grader) utilisé lors de l'évaluation, que vous pouvez utiliser pour tester vos solutions localement. L'évaluateur d'exemple lit les données depuis `stdin` et appelle la fonction `rybina_search` en utilisant le format d'entrée suivant.

L'entrée se compose de deux lignes :

- Ligne 1 : l'entier N .
- Ligne 2 : deux entiers S et T . Le premier, S , est la graine (seed) utilisée pour décider des inversions aléatoires ; le second, T , est la cible : le mot de passe initial AB écrit sous la forme d'un seul entier avec $0 \leq T < 10^{2N}$.

Après chaque tentative, l'évaluateur d'exemple inverse les deux moitiés du mot de passe de manière aléatoire en utilisant la graine S . (L'évaluateur utilisé lors de l'évaluation officielle est quant à lui adaptatif.)

L'évaluateur d'exemple affiche le résultat sur `stderr` : lorsque votre solution devine le mot de passe, il affiche le nombre de tentatives utilisées, sinon il affiche un court message indiquant la raison (une tentative invalide ou trop de tentatives).

Contraintes

- $1 \leq N \leq 9$.

Vous pouvez appeler la fonction `guess` au maximum 64 fois.

Score

Votre programme sera testé sur plusieurs tests, regroupés en sous-tâches. Pour obtenir les points d'une sous-tâche, votre programme doit résoudre correctement tous les tests qui lui sont associés.

- **Sous-tâche 0 [0 points]**: Cas de test d'exemple.
- **Sous-tâche 1 [5 points]**: $N = 1$ et $B = 0$, c'est-à-dire que la seconde moitié du mot de passe initial est 0.
- **Sous-tâche 2 [5 points]**: $N = 1$.
- **Sous-tâche 3 [10 points]**: $N \leq 4$ et Rybina n'inversera jamais le mot de passe.
- **Sous-tâche 4 [20 points]**: Rybina n'inversera jamais le mot de passe.
- **Sous-tâche 5 [15 points]**: Rybina inversera toujours le mot de passe.
- **Sous-tâche 6 [20 points]**: $N \leq 4$
- **Sous-tâche 7 [25 points]**: Aucune contrainte supplémentaire.

Exemples

Évaluateur	Solution
<code>rybina_search(2)</code>	
	<code>guess(1000)</code>
<code>return [1, 1]</code>	
	<code>guess(1000)</code>
<code>return [-1, 0]</code>	
	<code>guess(412)</code>
<code>return [0, 0]</code>	

Explication

Dans le premier exemple, $N = 2$ et les deux parties du mot de passe secret sont 12 et 04. La proposition 1000 est faite ; à ce moment-là, le mot de passe est 1204, donc la proposition est trop basse et, après que la proposition a été faite, Rybina signale que le mot de passe a été inversé. Le mot de passe est maintenant 412 et la même proposition de 1000 est faite. Cette fois, la proposition est trop élevée et Rybina signale également que le mot de passe n'a pas été inversé par la suite. Enfin, le mot de passe 412 est proposé, ce qui s'avère être correct, et le programme est terminé.